Mindful Kids: Master Project Report

1. Document Purpose

This is the single, complete reference document for this project.

It is written for:

- People with minimal technical background who need to understand the full system flow
- Developers who need a technical map of architecture, files, APIs, data flow, setup, and operations

This report covers:

- What the project does
- How to install and run it
- Complete directory and file map with purpose
- Full workflow from login to assessment, alerts, appointment booking, and doctor actions
- Core libraries, plugins, built-in APIs, and how they are used in this codebase
- Cleanup status for temporary/test scripts

2. Project Overview (Plain-Language)

Mindful Kids is a role-based web application for child mental well-being tracking.

Main users:

- Child: answers check-in questions and uses wellness activities
- Parent: creates child account, monitors risk, receives recommendations, books doctor appointments
- Doctor: reviews appointments and consented child assessment history, approves/rejects/completes appointments
- Admin: manages parent and doctor records and views high-level analytics

Important principles:

- It is a support and monitoring tool, not a diagnosis tool
- Parent consent is required before child assessment data is shared with doctors
- Access is controlled with role-based authentication

3. Technology Stack

3.1 Frontend

- React 18
- Vite (build + dev server)
- React Router v6 (routing and route protection)
- Tailwind CSS (utility-first styling)
- Recharts (charts)
- Lucide React (icon set)

3.2 Backend

- Node.js + Express
- SQLite via better-sqlite3
- JWT authentication via jsonwebtoken
- Password hashing via bcryptjs

- Email notifications via nodemailer (Gmail SMTP)
- dotenv for environment variables

4. Runtime Architecture

The application runs as two processes:

1. Frontend

- URL: http://localhost:3000
- Responsibilities: UI rendering, user interactions, route navigation, API calls

2. Backend

- URL: http://localhost:4000
- Responsibilities: auth, validation, business rules, database reads/writes, appointment notifications

Data source:

- SQLite database files under backend/data (primary DB and WAL/SHM sidecars)

5. Installation and Setup Guide

5.1 Prerequisites

Install:

- Node.js 18 or newer (Node 20 recommended)
- npm (comes with Node.js)

Verify installation:

- node -v
- npm -v

5.2 Clone / Open Project

Open this folder in VS Code:

- MENTAL-HEALTH-ASSISTANT-FOR-CHILDREN

5.3 Frontend Setup

From project root:

- npm install

5.4 Backend Setup

From backend folder:

- npm install

Create backend environment file:

- Copy backend/.env.example to backend/.env
- Configure values:
  - JWT_SECRET: any strong random string
  - EMAIL_USER: Gmail address used to send emails
  - EMAIL_PASS: Gmail app password
  - APP_URL: frontend URL (usually http://localhost:3000)

    - PORT: backend port (default 4000)

5.5 Start the App

Open two terminals.

Terminal A (root folder):

- npm run dev

Terminal B (backend folder):

- npm run dev or
- npm start

5.6 Build and Preview (Frontend)

From root:

- npm run build
- npm run preview

6. How to Run Day-to-Day (Quick Ops)

1. Start backend first.
2. Start frontend.
3. Open http://localhost:3000.
4. Use role-based login.
5. Validate backend health at http://localhost:4000/health.

7. Full Project Directory Map and File Uses

7.1 Root Directory

- index.html: Vite HTML entry shell
- package.json: frontend scripts and dependencies
- package-lock.json: dependency lock
- vite.config.js: Vite config, dev server port
- tailwind.config.js: Tailwind theme/content configuration
- postcss.config.js: PostCSS pipeline (Tailwind + Autoprefixer)
- README.md: project overview
- PROJECT_FLOW.md: workflow summary
- IMPLEMENTATION_GUIDE.md: implementation notes/documentation
- questions.pdf: questionnaire/support document asset
- .gitignore: git exclusions
- MASTER_PROJECT_REPORT.md: this master document

Directories:

- src: frontend source code
- public: static assets for frontend
- backend: backend API service
- data: additional SQLite DB files in root (runtime artifacts)
- Report: report documents (docx/pdf)
- dist: frontend production build output (generated)

- node_modules: installed dependencies (generated)

7.2 public

- vite.svg: sample/static icon asset

7.3 src (Frontend Source)

- App.jsx: route map and role-based route segmentation
- main.jsx: React entry point and providers mounting
- index.css: global styles + Tailwind imports

7.3.1 src/api

- api.js: centralized API client, token storage, request helpers, endpoint wrappers

7.3.2 src/components

- AdminDoctorsManagement.jsx: admin interface for doctor creation/editing and slot management
- AdminParentsManagement.jsx: admin interface for parent/child management
- AdminUserManagement.jsx: admin user management UI/actions
- AppointmentBooking.jsx: appointment booking component flow (legacy/current helper UI)
- AppointmentBookingModal.jsx: parent booking modal, consent checkbox, doctor/slot selection
- ProtectedRoute.jsx: route guard based on authentication and role

src/components/Layout

- AppLayout.jsx: sidebar/top-level app shell based on role
- ProtectedLayout.jsx: wrapper combining route protection and layout rendering

src/components/ui

- Card.jsx: reusable card container UI
- RiskBadge.jsx: risk-level badge styling component

7.3.3 src/contexts

- AuthContext.jsx: authentication state, login/logout/register, role access helpers
- AssessmentContext.jsx: local assessment persistence cache for UI
- ChildrenContext.jsx: child account fetch/create/delete orchestration

7.3.4 src/data

- mockData.js: static demo and mock dataset used in charts/UI

7.3.5 src/pages

- Alerts.jsx: parent/admin alert listing page
- AssessmentResults.jsx: assessment result rendering page
- DoctorManagementAdmin.jsx: doctor administration page variant/helper
- Login.jsx: role-aware login form
- ParentChildren.jsx: parent child-account management page
- Questionnaire.jsx: active child questionnaire flow with cooldown behavior
- QuestionnaireImproved.jsx: alternate/improved questionnaire variant
- Register.jsx: parent registration page
- Reports.jsx: parent/admin report views
- Settings.jsx: app settings page
- Wellness.jsx: wellness activities and streak UI

src/pages/dashboards

- AdminDashboard.jsx: analytics and management overview
- ChildDashboard.jsx: child home dashboard
- ParentDashboard.jsx: parent recommendations, appointment eligibility/locking logic
- DoctorDashboard.jsx: doctor appointment queue, patient records, status actions, forced password-change popup

7.3.6 src/utils

- mentalIllnessIdentification.js: logic/config for condition identification output
- riskAssessment.js: risk scoring and recommendation logic
- streakUtils.js: streak persistence and update utilities

7.4 backend

- package.json: backend scripts and dependencies
- package-lock.json: dependency lock
- README.md: backend documentation
- .env: runtime secrets/config (local only)
- .env.example: env template

7.4.1 backend/src

- server.js: Express app bootstrap, middleware, route registration
- db.js: SQLite schema creation and migrations
- seed.js: initial seed and backfill logic

backend/src/middleware

- auth.js: JWT signing, token verification, role checks

backend/src/routes

- auth.js: login/register, doctor password change, admin user auth endpoints
- parents.js: parent-child account CRUD and parent dashboard data
- assessments.js: assessment submission and retrieval with risk calculation
- alerts.js: parent alert retrieval
- doctors.js: doctor CRUD, slot CRUD, temp-password generation + onboarding email
- appointments.js: booking flow, consent checks, doctor actions, appointment status transitions
- admin.js: analytics and admin-level operations

backend/src/utils

- risk.js: backend risk calculation/recommendation definitions

7.4.2 backend/data

- mental-pro.db: SQLite database
- mental-pro.db-wal: SQLite write-ahead log sidecar
- mental-pro.db-shm: SQLite shared memory sidecar

7.5 data (Root)

- mental-pro.db
- mental-pro.db-wal
- mental-pro.db-shm

Note: root data and backend/data are runtime database artifact locations currently present in workspace.

7.6 Report

- PAPER.docx: project report document
- S8_FINAL_REPORT.docx: project report document
- S8_FINAL_REPORT.pdf: project report document

8. Cleanup Performed (Unwanted Files Removed)

The following temporary/test scripts were removed from backend root:

- test-odin-booking.mjs
- test-odin-email.mjs
- tmp-booking-test.mjs
- tmp-db-check.mjs
- tmp-send-test-email.js

This keeps backend root focused on production-relevant files.

9. Authentication, Authorization, and Security Model

9.1 Authentication

- JWT-based auth token is generated on successful login
- Token is stored in browser localStorage by frontend API client
- API requests attach Authorization: Bearer

9.2 Roles

Supported roles:

- child
- parent
- admin
- doctor

9.3 Authorization

- Route access controlled in frontend by ProtectedRoute and ProtectedLayout
- Backend validates token and role using middleware before sensitive actions

9.4 Password Security

- Passwords are hashed with bcryptjs
- Doctor creation auto-generates temporary password
- Doctor first login is flagged with mustChangePassword
- Forced password change popup appears until password is updated

10. Database Design Summary

Primary tables:

- users: parent/child/admin identity data and parent-child linkage
- doctors: doctor profile, credentials, first-login password-change flag
- time_slots: doctor availability slots
- assessments: child responses, risk level, recommendations, optional condition metadata
- alerts: parent-facing warning/monitoring messages

- appointments_updated: active appointment workflow table
- appointment_status_logs: audit trail for status changes
- appointments: legacy appointment table kept for compatibility

Key business constraints:

- Parent-child relation must match for child operations
- Appointment booking requires explicit consent flag
- Slot availability toggles with booking/status changes
- Child deletion preserves appointment history by nulling child_id and keeping child_name

11. API Surface and Behavior (Simplified + Technical)

11.1 Auth APIs

- POST /api/auth/register-parent
- POST /api/auth/login
- POST /api/auth/change-password (doctor only)
- GET /api/auth/users (admin)
- POST /api/auth/admin/create-parent (admin)
- DELETE /api/auth/admin/users/:userId (admin)

11.2 Parent/Child APIs

- GET /api/parents/:parentId/children
- POST /api/parents/:parentId/children
- DELETE /api/parents/:parentId/children/:childId
- GET /api/parents/:parentId/dashboard

11.3 Assessment APIs

- POST /api/children/:childId/assessments
- GET /api/children/:childId/assessments

11.4 Alert APIs

- GET /api/alerts?parentId=

11.5 Doctor and Slot APIs

- GET /api/doctors
- GET /api/doctors/:doctorId/slots
- POST /api/doctors (admin)
- PUT /api/doctors/:doctorId (admin)
- DELETE /api/doctors/:doctorId (admin)
- GET /api/slots/available
- POST /api/slots (admin or doctor self-management)
- DELETE /api/slots/:slotId (admin or owning doctor)

11.6 Appointment APIs

- POST /api/appointments/book
- GET /api/appointments/parent/:parentId
- GET /api/appointments/admin/all
- GET /api/appointments/doctor/me
- GET /api/appointments/doctor/patients
- PUT /api/appointments/:appointmentId/confirm

- DELETE /api/appointments/:appointmentId
- POST /api/appointments (legacy endpoint)

11.7 Admin APIs

- GET /api/admin/analytics
- DELETE /api/admin/users/:userId
- PUT /api/admin/appointments/:appointmentId/status

12. Complete Workflow (End-to-End)

12.1 Parent Registration and Child Creation

1. Parent registers from frontend.
2. Backend creates parent row in users table.
3. Parent logs in and receives JWT.
4. Parent creates child account.
5. Backend validates parent ownership, hashes child password, creates child row linked by parent_id.

12.2 Child Assessment Flow

1. Child logs in with username + password.
2. Child opens questionnaire and answers grouped questions.
3. Frontend submits responses to assessment API.
4. Backend calculates avg score and risk level.
5. Backend stores assessment record.
6. If risk is moderate/high, backend inserts alert for parent.
7. Parent dashboard and reports reflect latest assessment.

12.3 Parent Appointment Booking Flow

1. Parent opens booking modal for a child.
2. Parent selects doctor and available slot.
3. Parent must check consent checkbox to share child data.
4. Backend validates:
    - parent-child ownership
    - slot availability
    - consent flag
    - duplicate booking conflict
5. Backend inserts booked appointment and marks slot unavailable.
6. Backend sends doctor email with confirm/reject deep links.
7. Parent dashboard blocks new booking for same child while active status exists (booked/confirmed/pending/postponed).

12.4 Doctor Approval/Decision Flow

1. Doctor logs in with email + password.
2. If temp password account: forced popup requests immediate password change.
3. Doctor dashboard shows queued appointments and consented patients.
4. Doctor updates status to confirmed/rejected/postponed/completed.
5. Backend writes status log and updates appointment row.
6. Slot release rules:
    - rejected or completed -> slot released
    - confirmed -> slot remains unavailable

7. Parent booking lock for child clears when active appointment is no longer active.

12.5 Admin Operations Flow

1. Admin manages parent and doctor accounts.
2. Admin can create doctor with auto temporary password + onboarding email.
3. Admin can add slots and review analytics.
4. Analytics uses aggregated DB queries for risk distribution, appointment status counts, and condition breakdown.

13. Frontend State and Data Handling

- AuthContext manages authenticated user state and role checks.
- ChildrenContext fetches backend children and handles child CRUD sync.
- AssessmentContext keeps a local assessment cache for UI responsiveness (in addition to backend persistence flow).
- API module centralizes fetch logic and error normalization.

14. Built-In APIs and Plugins Used (How They Work Here)

14.1 Browser/Web APIs

- fetch: frontend HTTP calls to backend REST endpoints
- localStorage: stores auth token, auth user object, and some local context/state
- Date and Intl/JS date handling: appointment timing and display formatting
- JSON.parse / JSON.stringify: transport and persistence serialization

14.2 React Built-Ins

- useState: local component state
- useEffect: lifecycle and side effects (fetching data on mount/dependency changes)
- useMemo: optimized derived values (filters, grouped lists)
- useCallback: stable callback references
- Context API (createContext/useContext): cross-component shared state

14.3 Vite and Build Plugins

- Vite: fast dev server and production bundler
- @vitejs/plugin-react: JSX transform, fast refresh, React support in Vite
- PostCSS: CSS transformation pipeline
- Tailwind CSS plugin: utility class generation from content scan
- Autoprefixer: auto vendor-prefixing CSS for broader browser compatibility

14.4 Backend Libraries

- Express: route handlers + middleware chain
- better-sqlite3: synchronous SQL operations with prepared statements
- jsonwebtoken: token signing/verification
- bcryptjs: password hashing and verification
- nodemailer: SMTP email delivery for doctor notifications and onboarding
- dotenv: environment variable loading from .env

14.5 UI/Data Libraries

- react-router-dom: route definitions, navigation, protected navigation
- recharts: charts on dashboards/reports

- lucide-react: icon components used across pages

15. Error Handling and Observability

- Backend routes return HTTP status + JSON messages for invalid input and auth failures.
- Central backend error middleware catches unhandled errors and returns generic 500 response.
- SMTP transporter verification logs mail setup health.
- Frontend catches API errors and surfaces readable messages in UI components.

16. Environment Variables and Configuration

Backend (.env):

- JWT_SECRET
- EMAIL_USER
- EMAIL_PASS
- APP_URL
- PORT
- DB_PATH (optional override)

Frontend:

- VITE_API_URL (optional; defaults to http://localhost:4000/api)

17. Deployment Notes

Minimum deployment requirements:

- Node.js runtime for frontend build and backend API
- Persistent writable storage for SQLite database file
- Environment variables configured in deployment target
- SMTP credentials for email features

Recommended production improvements:

- Replace SQLite with managed DB for high-scale concurrency
- Add HTTPS + secure cookies/token strategy
- Centralized logging and monitoring
- Add API tests and UI tests in CI pipeline

18. Known Design Characteristics

- AssessmentContext uses localStorage caching for frontend convenience while backend also persists data.
- Legacy appointments endpoint/table still exists for backward compatibility.
- Current model prioritizes appointment workflow through appointments_updated.
- Multiple DB artifact locations exist in workspace (root data and backend/data).

19. Operator Checklist (Practical)

Before demo/run:

1. Confirm backend/.env values.
2. Start backend and verify /health endpoint.
3. Start frontend.
4. Verify login for each role.

5. Validate booking and doctor status change paths.
6. Verify forced doctor password-change popup on first login.

20. Summary

This project implements a full role-driven child well-being monitoring platform with:

- account management,
- questionnaire-based risk scoring,
- parent alerts and recommendations,
- consent-gated appointment workflow,
- doctor triage and completion actions,
- analytics for administration,
- and a secure first-login password-change flow for doctor accounts.

This document can be used as the master reference for understanding, onboarding, operation, and technical maintenance.